# generalManagement MQTT/TLS/JSON

## Interface Design Description

Service ID: *"generalManagement"*

**Abstract**
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the DynamicServiceOrchestration system.

# Contents

**1 Overview** **3**

**2 Service Operations** **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 5

**3 Data Models** **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

**4 References** **9**

**5 Revision History** **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

# 1 Overview

This document describes the `generalManagement` service interface of the DynamicServiceOrchestration system over MQTT, which provides general management of the DynamicServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

| Profile type | Type | Version |
|---|---|---|
| Transfer protocol | MQTT | 3.1.1 [1] |
| Data encryption | TLS | 1.3 [2], 1.2 [3] |
| Encoding | JSON | RFC 8259 [4] |
| Compression | N/A | - |
| Semantics | N/A | - |
| Ontology | N/A | - |

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

# 2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

| Operation | Request topic |
|---|---|
| `get-log` | `arrowhead/serviceorchestration/general/management/get-log` |
| `get-config` | `arrowhead/serviceorchestration/general/management/get-config` |

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

## 2.1 PUBLISH `arrowhead/serviceorchestration/general/management/get-log`

**Operation:** get-log
**Input:** LogRequest
**Output:** LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

## 2.2 PUBLISH `arrowhead/serviceorchestration/general/management/get-config`

**Operation:** get-config
**Input:** Array<String>
**Output:** KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

# 3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

## 3.1 struct MqttRequestTemplate

The request envelope.

| Object Field | Value Type | Description |
|---|---|---|
| "traceId" | String | Identifier chosen by the requester, returned in the response. |
| "authentication" | String | Identity of the requester (see Section 2). |
| "responseTopic" | String | Topic on which the response is published. |
| "qosRequirement" | Number | MQTT QoS of the response (0, 1 or 2). |
| "params" | Object<String> | Optional parameters (the query parameters of the HTTP binding). |
| "payload" | Object | The operation input. |

## 3.2 struct MqttResponseTemplate

The response envelope.

| Object Field | Value Type | Description |
|---|---|---|
| "status" | Number | Status code. |
| "traceId" | String | The trace identifier of the request. |
| "receiver" | String | Name of the requester. |
| "payload" | Object | The operation output, or an ErrorMessage. |

## 3.3 struct ErrorMessage

| Object Field | Value Type | Description |
|---|---|---|
| "errorMessage" | String | |
| "errorCode" | Number | |
| "exceptionType" | ExceptionType | |
| "origin" | String | |

## 3.4 struct KeyValues

Configuration values of the system.

| Object Field | Value Type | Description |
|---|---|---|
| "map" | Object<String> | The configuration keys and values; secrets are not returned. |

## 3.5 struct LogEntry

One log entry.

| Object Field | Value Type | Description |
|---|---|---|
| "logId" | String | Identifier of the log entry. |
| "entryDate" | DateTime | UTC time of the log entry. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |
| "severity" | String | Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level. |
| "message" | String | Text of the log entry or status message of the job. |
| "exception" | String | Stack trace or exception text of the log entry, if any. |

## 3.6 struct LogEntryListResponse

List of log entries.

| Object Field | Value Type | Description |
|---|---|---|
| "entries" | Array<LogEntry> | Matching records of the current page. |
| "count" | Number | Total number of matching records (all pages). |

## 3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

| Object Field | Value Type | Description |
|---|---|---|
| "pagination" | Page | Paging and sorting of the result. |
| "from" | DateTime | Return entries from this UTC time. |
| "to" | DateTime | Return entries until this UTC time. |
| "severity" | String | Minimum log level to return. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |

## 3.8 struct Page

Paging and sorting; all fields optional.

| Object Field | Value Type | Description |
|---|---|---|
| "page" | Number | Zero-based page number. |
| "size" | Number | Number of records per page (at most `max.page.size`, default 1000). |
| "direction" | String | Sort direction, `ASC` or `DESC`. |
| "sortField" | String | Field to sort by. |

## 3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

| Type | Description |
|---|---|
| Array | A JSON Array [4]; Array<A> has elements of type A. |
| DateTime | JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`. |
| ExceptionType | One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String). |
| Number | JSON Number, an integer. |
| Object | JSON Object; Object<A> has string keys and values of type A. |
| String | JSON String. |

# 4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

# 5 Revision History

## 5.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-10-22 | 5.2.2 | Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace. | Jerker Delsing |

## 5.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.2.2 | |